CS 251 — Computer Organization and Design

Vishesh Gupta

University of Waterloo

Contents

1 Introduction **1**
- 1.1 Course Information (Winter 2017) . . . 1
- 1.2 Course Outline . . . 1
- 1.3 Gist of the Course . . . 2
- 1.4 Instruction Set Architecture . . . 2
- 1.5 The Big Picture . . . 2
- 1.6 Assembly Language and MIPS . . . 3
  - 1.6.1 Registers . . . 4
  - 1.6.2 Instruction addresses . . . 4
  - 1.6.3 The three MIPS instruction formats . . . 4
  - 1.6.4 Jumping counts words, not bytes . . . 5

2 MIPS Basics: Five Instructions **6**
- 2.1 The Machine Model . . . 6
- 2.2 The Five Instructions . . . 6
  - 2.2.1 Arithmetic: `add` and `sub` . . . 6
  - 2.2.2 Memory: `lw` and `sw` . . . 7
  - 2.2.3 Branching: `beq` . . . 7
- 2.3 Writing Programs with Five Instructions . . . 7
- 2.4 Machine Encoding Preview . . . 8

3 Digital Logic: Combinational Circuits **9**
- 3.1 Boolean Algebra . . . 9
  - 3.1.1 Laws . . . 9
- 3.2 Gates . . . 10
- 3.3 Truth Tables and Canonical Forms . . . 10
- 3.4 Karnaugh Maps . . . 11
- 3.5 Combinational Building Blocks . . . 11
  - 3.5.1 Multiplexer . . . 11
  - 3.5.2 Decoder . . . 12
  - 3.5.3 Adders . . . 12
  - 3.5.4 The ALU . . . 13
- 3.6 Timing . . . 13

4 Digital Logic: Sequential Circuits **15**
- 4.1 Latches . . . 15
  - 4.1.1 SR latch . . . 15

4.1.2 D latch . . . 15
4.2 Clocks and Flip-Flops . . . 16
4.2.1 Other flip-flops . . . 16
4.3 Registers and Register Files . . . 16
4.4 Finite State Machines . . . 17
4.5 Memory . . . 18

5 Data Representation and Manipulation 19
5.1 Number Bases . . . 19
5.2 Unsigned Integers . . . 19
5.3 Signed Integers . . . 19
5.3.1 Sign–magnitude and one's complement . . . 19
5.3.2 Two's complement . . . 20
5.3.3 Sign extension . . . 20
5.3.4 Signed overflow . . . 20
5.4 Multiplication and Division . . . 20
5.5 Shifts and Logical Operations . . . 21
5.6 Floating Point: IEEE 754 . . . 21
5.6.1 Special values (single precision) . . . 21
5.6.2 Floating-point arithmetic . . . 22
5.7 Characters . . . 22
5.8 Endianness . . . 22

6 The MIPS Instruction Set in Depth 23
6.1 Design Principles . . . 23
6.2 Registers and Conventions . . . 23
6.3 Instruction Formats . . . 24
6.3.1 Common instructions . . . 24
6.3.2 Large constants . . . 24
6.4 Addressing Modes . . . 24
6.5 Control Flow . . . 25
6.6 Procedures . . . 25
6.6.1 The stack . . . 26
6.6.2 Memory layout . . . 26
6.7 From Source to Execution . . . 26
6.8 RISC vs CISC . . . 26

7 The Single-Cycle Datapath and Control 28
7.1 Performance . . . 28
7.2 Datapath Elements . . . 28
7.3 Building the Datapath . . . 28
7.4 Control . . . 29
7.4.1 Main control table . . . 29
7.4.2 ALU control . . . 30
7.4.3 Adding jump . . . 30
7.5 Timing of the Single-Cycle Design . . . 30

8 The Multi-Cycle Datapath **32**
8.1 Key Ideas . . . 32
8.2 The Five Steps . . . 33
8.3 Control as an FSM . . . 33
8.4 Performance Comparison . . . 34
8.5 Exceptions (brief) . . . 35

9 Pipelining and Hazards **36**
9.1 The Five-Stage MIPS Pipeline . . . 36
9.1.1 Performance . . . 36
9.2 Hazards . . . 37
9.2.1 Structural hazards . . . 37
9.2.2 Data hazards . . . 37
9.2.3 Control hazards . . . 38
9.3 Beyond the Basic Pipeline . . . 39

10 Memory Hierarchy and Caches **41**
10.1 Locality . . . 41
10.2 Terminology . . . 41
10.3 Direct-Mapped Caches . . . 42
10.3.1 Block size . . . 42
10.4 The Three Cs of Misses . . . 42
10.5 Associative Caches . . . 42
10.6 Writes . . . 43
10.7 Cache Performance . . . 43

11 Virtual Memory **45**
11.1 Why Virtual Memory? . . . 45
11.2 Pages and Address Translation . . . 45
11.3 Page Tables . . . 45
11.4 Page Faults . . . 46
11.5 The TLB . . . 46
11.6 Caches and Virtual Memory Together . . . 47
11.7 Protection . . . 47

12 ARM: 32-bit and ARMv8 **49**
12.1 ARM 32-bit (ARMv7) . . . 49
12.1.1 Registers . . . 49
12.1.2 Condition codes and conditional execution . . . 49
12.1.3 Flexible second operand . . . 50
12.1.4 Addressing modes . . . 50
12.1.5 Procedures . . . 50
12.1.6 Thumb . . . 51
12.2 Comparison: MIPS vs ARM 32-bit . . . 51
12.3 ARMv8 (AArch64) . . . 51
12.4 Fallacies and Pitfalls (course themes) . . . 52

Chapter 1

Introduction

1.1 Course Information (Winter 2017)

- Instructor: Rosina Kharal
- Classroom: RCH 305, Section 001, 8:30 AM–9:50 AM
- Midterm: February 8, 2017. Final exam: TBA.

Recommended texts.

1. Patterson and Hennessy, *Computer Organization & Design: The Hardware/Software Interface*, 5th edition, Morgan Kaufmann, 2013 (ISBN 0124077269).
2. (Optional) Harris and Harris, *Digital Design & Computer Architecture*, 2nd edition, Morgan Kaufmann, 2012 (ISBN 0123944244).

1.2 Course Outline

1. MIPS basic introduction (five instructions)
2. Digital logic design
3. Data representation and manipulation
4. Designing a datapath
5. Single-cycle datapath and multi-cycle datapath
6. Pipelined datapath and pipeline hazards
7. Memory hierarchies (caches and virtual memory)
8. ARM: 32-bit (and a look at ARMv8)

Exam note

A major part of the final exam covers the last part of the course (pipelining, memory hierarchy, ARM), so do not neglect it.

1.3 Gist of the Course

This is a *processor* course: it is about how to build a processor and how it works. We start from the language the processor speaks (its *instruction set*), build the hardware building blocks from logic gates, and then assemble those blocks into a working datapath that executes the instructions. Finally we look at the techniques that make real processors fast: pipelining and the memory hierarchy.

1.4 Instruction Set Architecture

We write programs in many high-level languages (C, C++, Racket, ...), but the processor understands only one language, a low-level *machine language*. A compiler translates high-level code to assembly, and an assembler translates assembly into binary machine code.

Definition 1.1. An **instruction** is a single operation the processor knows how to perform, for example `add`, `sub`, a branch or a jump. The **instruction set architecture (ISA)** is the complete set of instructions, registers, data types and memory model that the hardware exposes to software. It is the contract between hardware and software.

The ISA is an abstraction: two processors can implement the same ISA very differently (one simple and slow, one pipelined and fast) and still run the same binary programs.

1.5 The Big Picture

Hardware is designed according to a few recurring principles (simplicity favours regularity, smaller is faster, make the common case fast, good design demands good compromises). Every computer, from a phone to a server, has the same five classic components:

Computer Organization: The Big Picture

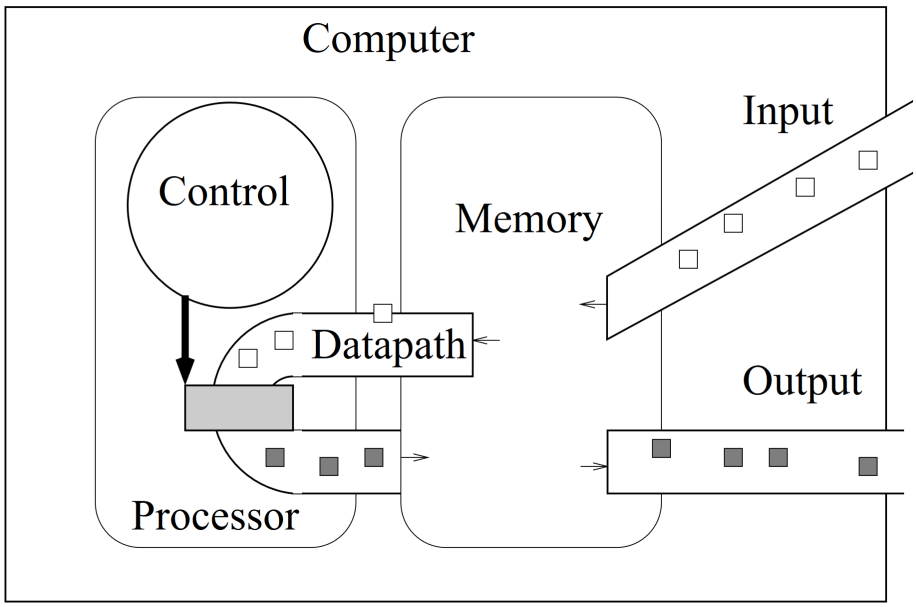


Figure 1.1: The five classic components of a computer: control and datapath (together the processor), memory, input and output.

- **Datapath**: the hardware that actually performs operations on data (registers, ALU, multiplexers, buses). Formally, the datapath is the collection of hardware units that come together to execute an instruction.
- **Control**: tells the datapath, memory and I/O what to do according to the current instruction.
- **Memory**: holds both programs (instructions) and data while they are in use (stored-program concept).
- **Input** and **Output**: devices that bring data in and send results out.

1.6 Assembly Language and MIPS

Assembly language is a human-readable form of machine code. Each line corresponds (essentially) to one machine instruction. Different operations have different names (mnemonics) and different *formats*.

Definition 1.2. MIPS (originally "Microprocessor without Interlocked Pipeline Stages") is a **RISC** (Reduced Instruction Set Computer) architecture:

1. it provides only very basic operations, each doing a small amount of work;
2. all instructions are the same size: in this course, a 32-bit word (64-bit versions of MIPS exist but we use the 32-bit one);
3. the first six bits of every instruction are the **opcode**.

The execution of an instruction in hardware is divided into five steps: instruction fetch, instruction decode/register read, execute (ALU), memory access, and register write-back. This five-way split will reappear when we pipeline the processor.

1.6.1 Registers

Definition 1.3. A **register** is a small, very fast storage location inside the processor. It plays a role similar to a temporary variable in a high-level language. MIPS has 32 general-purpose registers, each 32 bits wide.

Example 1.4. Compile `f = (g + h) - (i + j);` where the variables f, g, h, i, j live in registers `$s1`, ..., `$s5`:

```
add $t0, $s2, $s3    # t0 = g + h
add $t1, $s4, $s5    # t1 = i + j
sub $s1, $t0, $t1    # f  = t0 - t1
```

Since an instruction has only two source operands, we need temporaries (`$t0`, `$t1`) to hold intermediate results.

Two issues already show up in this tiny example:

- **Data dependency**: the `sub` needs the results of both `add`s, so it cannot run before them. This matters a lot once instructions overlap in a pipeline.
- **Overflow**: a 32-bit register cannot hold every sum; adding two large numbers may produce a result that does not fit.

1.6.2 Instruction addresses

Instructions are stored in memory one after the other. Since every MIPS instruction is 4 bytes, consecutive instructions are at addresses $0, 4, 8, 12, \ldots$:

Address	Instruction
0	`add $3, $2, $1`
4	`sub ...`
8	`sub ...`
12	`sub ...`

1.6.3 The three MIPS instruction formats

The *format* of an instruction describes how many operands it has and of what type.

- **R-format** (register): `add $1, $2, $3` adds the contents of `$2` and `$3` and stores the result in `$1`. Often written `add rd, rs, rt` where `rd` is the destination register.
- **I-format** (immediate): `addi $1, $2, 100` adds the constant 100 to `$2` and stores the result in `$1`.
- **J-format** (jump): `j 28`, used for unconditional jumps (details later).

1.6.4 Jumping counts words, not bytes

When we say "jump to instruction 40" we mean the 40th instruction (word), so the program counter moves to byte address $40 \times 4 = 160$. Because every instruction is 4 bytes, the low two bits of an instruction address are always 00, and MIPS saves encoding space by storing word offsets and multiplying by 4 in hardware.

Summary

- CS 251 is about how a processor is built: ISA, digital logic, datapath and control, pipelining, memory hierarchy.
- The ISA is the hardware/software contract; MIPS is a simple 32-bit RISC ISA with fixed-length instructions and a 6-bit opcode.
- A computer = datapath + control (processor) + memory + input + output.
- Registers are fast local storage; MIPS has 32 of them.
- Three formats: R (three registers), I (two registers + constant), J (jump target). Instruction addresses go up by 4.

Chapter 2

MIPS Basics: Five Instructions

Before building any hardware, we need to know what the hardware is supposed to do. For most of the course we use a tiny subset of MIPS consisting of five instructions. They are enough to write real (if clumsy) programs and they exercise every part of the datapath we will later build.

2.1 The Machine Model

- 32 registers `$0`–`$31`, each holding a 32-bit **word**. Register `$0` always reads as 0; writes to it are ignored.
- A **program counter** (PC) holding the address of the current instruction.
- A byte-addressed memory of 2^{32} bytes. Words are 4 bytes and word addresses must be multiples of 4 (*alignment*).
- MIPS is a **load/store architecture**: arithmetic works only on registers; memory is touched only by loads and stores.

2.2 The Five Instructions

Instruction	Meaning	Format
`add $d, $s, $t`	$\$d \leftarrow \$s + \$t$	R
`sub $d, $s, $t`	$\$d \leftarrow \$s - \$t$	R
`lw $t, i($s)`	$\$t \leftarrow \text{MEM}[\$s + i]$	I
`sw $t, i($s)`	$\text{MEM}[\$s + i] \leftarrow \$t$	I
`beq $s, $t, i`	if $\$s = \$t$ then $PC \leftarrow PC + 4 + 4i$	I

In every case, after the instruction (unless a branch is taken) the PC advances by 4. Here i is a 16-bit signed constant (immediate).

2.2.1 Arithmetic: `add` and `sub`

Both take three registers. The order of operands in assembly is destination first. There is no "move" instruction; to copy a register use `add $d, $s, $0`.

2.2.2 Memory: `lw` and `sw`

The address is computed as *base register + signed offset*. This "displacement" addressing mode is natural for arrays and structs:

Example 2.1. Suppose `$1` holds the address of an array `A` of 32-bit integers. Then `A[3]` is at byte offset $3 \times 4 = 12$:

```
lw  $2, 12($1)      # $2 = A[3]
add $2, $2, $2      # double it
sw  $2, 12($1)      # A[3] = $2
```

Pitfall: byte offsets

Offsets are in *bytes*, not elements. `A[k]` for word arrays is at offset $4k$. Forgetting the factor of 4 is the most common beginner error.

2.2.3 Branching: `beq`

`beq` compares two registers. If equal, it jumps i *instructions* relative to the instruction *after* the branch (because the PC has already been incremented by 4 when the branch is evaluated). So `beq $0, $0, -1` is an infinite loop and `beq $0, $0, 0` does nothing useful.

2.3 Writing Programs with Five Instructions

Example 2.2 (If statement)**.** Compute `if (a == b) c = a + b; else c = a - b;` with `a, b, c` in `$1, $2, $3`.

```
      beq $1, $2, 2      # if a == b skip the next two instructions
      sub $3, $1, $2     # else part
      beq $0, $0, 1      # unconditional "jump" over the then part
      add $3, $1, $2     # then part
                         # continue here
```

`beq $0, $0, k` is the standard trick for an unconditional branch, since `$0 = $0` is always true.

Example 2.3 (Loop: sum an array)**.** `$1` = address of array, `$2` = number of elements n, `$4` holds the constant 4 and `$5` holds the constant 1 (assume they were set up for us, since we have no immediate arithmetic yet). Result in `$3`.

```
      add $3, $0, $0     # sum = 0
loop: beq $2, $0, 5      # if n == 0 exit (skip 5 instructions)
      lw  $6, 0($1)      # t = *p
      add $3, $3, $6     # sum += t
      add $1, $1, $4     # p += 4  (next word)
      sub $2, $2, $5     # n -= 1
      beq $0, $0, -6     # back to loop
                         # exit
```

Counting the branch offset: from the instruction after `beq $0,$0,-6`, we must go back 6 instructions to reach `loop`.

2.4 Machine Encoding Preview

Every instruction is 32 bits. The first 6 bits are the opcode.

R	op (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	op (6)	rs (5)	rt (5)	immediate (16)		

Instruction	opcode	funct
`add`	000000	100000 (0x20)
`sub`	000000	100010 (0x22)
`lw`	100011 (0x23)	–
`sw`	101011 (0x2B)	–
`beq`	000100 (0x04)	–

Example 2.4. Encode `add $3, $1, $2`: op = 0, rs = 1, rt = 2, rd = 3, shamt = 0, funct = 0x20.

$$\underbrace{000000}_{op}\ \underbrace{00001}_{rs}\ \underbrace{00010}_{rt}\ \underbrace{00011}_{rd}\ \underbrace{00000}_{shamt}\ \underbrace{100000}_{funct} = \texttt{0x00221820}.$$

Note that in assembly the destination comes first, but in the encoding `rd` comes third.

Example 2.5. Encode `lw $5, -8($29)`: op = 0x23, rs = 29, rt = 5, imm = -8 = `0xFFF8`. Bits: `100011 11101 00101 1111111111111000` = `0x8FA5FFF8`.

Summary

- Five instructions: `add`, `sub`, `lw`, `sw`, `beq`. Two R-format, three I-format.
- Memory addresses are base + signed byte offset; words are aligned.
- Branch offsets count instructions relative to $PC + 4$.
- `beq $0, $0, k` acts as an unconditional branch.
- These five instructions drive the design of the single-cycle datapath later in the course.

Chapter 3

Digital Logic: Combinational Circuits

A processor is built out of transistors acting as switches. We abstract transistors into *logic gates*, gates into *combinational circuits* (whose outputs depend only on current inputs), and later add memory to get *sequential circuits*.

3.1 Boolean Algebra

Signals take one of two values, 0 (false, low voltage) and 1 (true, high voltage). The basic operations are

- AND: $A \cdot B$ (or AB), 1 iff both are 1;
- OR: $A + B$, 1 iff at least one is 1;
- NOT: $\overline{A}$ (or A'), flips the bit.

Derived operations: NAND $\overline{AB}$, NOR $\overline{A+B}$, XOR $A \oplus B = A\overline{B} + \overline{A}B$, XNOR $\overline{A \oplus B}$.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

3.1.1 Laws

Law	AND form	OR form
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$AA = A$	$A + A = A$
Complement	$A\overline{A} = 0$	$A + \overline{A} = 1$
Commutative	$AB = BA$	$A + B = B + A$
Associative	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
Absorption	$A(A + B) = A$	$A + AB = A$
De Morgan	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}\,\overline{B}$

Note the second distributive law $A+BC=(A+B)(A+C)$, which has no analogue in ordinary arithmetic. Every law has a *dual* obtained by swapping AND/OR and 0/1.

Example 3.1. Simplify $F=\overline{A}B+AB+A\overline{B}$.

$$F=(\overline{A}+A)B+A\overline{B}=B+A\overline{B}=(B+A)(B+\overline{B})=A+B.$$

3.2 Gates

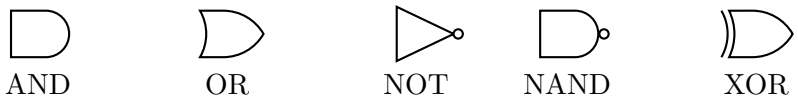


Figure 3.1: Standard (US) gate symbols. A bubble on an output means inversion.

Universality

NAND alone is *functionally complete*: $\overline{A}=\overline{AA}$, $AB=\overline{\overline{AB}}$, $A+B=\overline{\overline{A}\,\overline{B}}$. The same is true for NOR. Real CMOS hardware prefers NAND/NOR because they need fewer transistors than AND/OR.

3.3 Truth Tables and Canonical Forms

Any Boolean function of n inputs is described by a truth table with 2^n rows.

Definition 3.2. A **minterm** is a product (AND) containing every variable once, either plain or complemented; it is 1 in exactly one row. The **sum of products (SOP)** form is the OR of the minterms for rows where $F=1$. Dually, a **maxterm** is a sum that is 0 in exactly one row, and the **product of sums (POS)** form ANDs the maxterms of the rows where $F=0$.

Example 3.3 (Majority function)**.** $F(A,B,C)=1$ iff at least two inputs are 1.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

SOP: $F=\overline{A}BC+A\overline{B}C+AB\overline{C}+ABC=\sum m(3,5,6,7)$. This simplifies to $F=AB+AC+BC$.

3.4 Karnaugh Maps

A K-map arranges the truth table in a grid whose rows and columns follow **Gray code** order (00, 01, 11, 10) so that physically adjacent cells differ in exactly one variable. Adjacent 1s can then be combined, since $XY + X\overline{Y} = X$.

Rules for grouping:

1. Groups must be rectangles of size $1, 2, 4, 8, \ldots$ (powers of 2).
2. Groups may wrap around the edges (the map is a torus).
3. Make groups as large as possible, and use as few as possible; every 1 must be covered; overlapping is allowed.
4. Don't-care entries (X) may be included in a group if that helps, but need not be covered.

Each group of 2^k cells eliminates k variables.

Example 3.4. Majority function K-map (rows A, columns BC):

$A\backslash BC$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Three groups of two: the column $BC = 11$ gives BC; cells $(1, 01), (1, 11)$ give AC; cells $(1, 11), (1, 10)$ give AB. So $F = AB + AC + BC$.

Example 3.5 (Four variables)**.** $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$.

$AB\backslash CD$	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

The four corners form one group (wrap-around) giving $\overline{B}\,\overline{D}$; the middle 2×2 block gives BD. Hence $F = \overline{B}\,\overline{D} + BD = \overline{B \oplus D}$.

3.5 Combinational Building Blocks

3.5.1 Multiplexer

A **multiplexer (mux)** selects one of several inputs using select lines. A 2-to-1 mux with inputs I_0, I_1 and select S:

$$Y = \overline{S} I_0 + S I_1.$$

A 2^k-to-1 mux needs k select bits. Muxes are everywhere in the datapath: whenever a value may come from two places, a mux chooses.

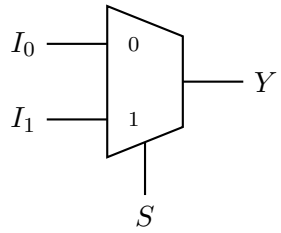


Figure 3.2: 2-to-1 multiplexer.

A mux of n-bit buses is just n one-bit muxes sharing the select line.

3.5.2 Decoder

An n-to-2^n **decoder** has n inputs and 2^n outputs; exactly the output whose index equals the input value is 1. A 2-to-4 decoder: $Y_0 = \overline{A_1}\,\overline{A_0}$, $Y_1 = \overline{A_1}A_0$, $Y_2 = A_1\overline{A_0}$, $Y_3 = A_1A_0$. Decoders select a register in the register file and a row in memory. An **encoder** does the reverse.

3.5.3 Adders

Definition 3.6. A **half adder** adds two bits: $S = A \oplus B$, $C_{out} = AB$. A **full adder** also takes a carry-in:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + AC_{in} + BC_{in}.$$

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Notice C_{out} is exactly the majority function.

Ripple-carry adder. Chain n full adders, the carry-out of bit i feeding the carry-in of bit $i+1$. Simple, but the delay is $\Theta(n)$ because the carry may have to ripple through every stage.

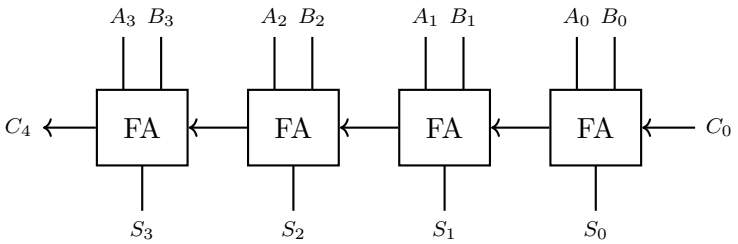


Figure 3.3: 4-bit ripple-carry adder.

Carry-lookahead (idea). Define *generate* $g_i = A_i B_i$ and *propagate* $p_i = A_i + B_i$. Then $c_{i+1} = g_i + p_i c_i$, which can be expanded, e.g. $c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$, so all carries are computed in parallel in a constant number of gate levels (at the cost of more gates). Hierarchical lookahead gives $\Theta(\log n)$ delay.

Subtraction. $A - B = A + \overline{B} + 1$ in two's complement: invert B with XOR gates and set $C_0 = 1$. A single control bit (`Binvert`) turns an adder into an adder/subtractor.

3.5.4 The ALU

The **arithmetic logic unit** computes one of several operations on two 32-bit inputs, chosen by a control input. A 1-bit ALU slice contains an AND gate, an OR gate and a full adder, with a mux choosing the result; 32 slices are chained through the carries.

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

The 4-bit control is `Ainvert`, `Bnegate` (= `Binvert` and C_0), and a 2-bit operation select. Extra outputs:

- **Zero**: 1 if the result is 0 (NOR of all result bits); used by `beq` (subtract and test Zero).
- **Overflow**: for signed add, $C_n \oplus C_{n-1}$ (carry into the sign bit differs from carry out).
- **Set on less than**: compute $A - B$; the sign bit of the result (corrected for overflow) is routed to bit 0 of the output, all other bits 0.

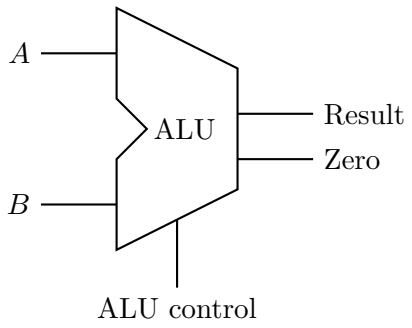


Figure 3.4: ALU symbol.

3.6 Timing

Gates have *propagation delay*. The delay of a combinational circuit is the delay along its **critical path** (longest path from any input to any output). This will determine the clock period of the processor.

Summary

- Boolean algebra laws (especially De Morgan) let us manipulate expressions; NAND and NOR are universal.
- Truth table → SOP via minterms → simplify with K-maps (Gray code, power-of-two groups, wrap-around, don't cares).
- Building blocks: mux (select), decoder (one-hot), full adder, ripple-carry and carry-lookahead adders, ALU with Zero and Overflow.
- Subtraction = add the complement plus 1.

Chapter 4

Digital Logic: Sequential Circuits

Combinational circuits have no memory: outputs are a function of current inputs only. To build registers, the PC and memory, we need circuits whose outputs also depend on their *state*. These are **sequential circuits**. The key ingredient is feedback.

4.1 Latches

4.1.1 SR latch

Two cross-coupled NOR gates form an **SR (set–reset) latch**.

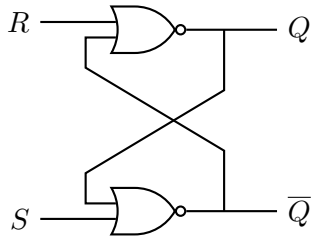


Figure 4.1: SR latch from two NOR gates.

S	R	Q_{next}	
0	0	Q	hold (remember)
0	1	0	reset
1	0	1	set
1	1	–	forbidden ($Q = \overline{Q} = 0$, unpredictable after)

4.1.2 D latch

To avoid the forbidden state, a **D latch** derives $S = D \cdot C$ and $R = \overline{D} \cdot C$ from a data input D and an enable/clock C. When $C = 1$ the latch is *transparent* (Q follows D); when $C = 0$ it holds its value.

4.2 Clocks and Flip-Flops

A **clock** is a periodic signal. Its *period* is the time of one cycle; the frequency is the reciprocal (a 1 ns period means 1 GHz). In **edge-triggered** design, state changes only at a clock edge (we use the rising edge), so all state elements update simultaneously and combinational logic has a whole cycle to settle in between.

Definition 4.1. A **D flip-flop** samples D at the rising clock edge and holds that value in Q until the next rising edge. It can be built from two D latches in a *master–slave* arrangement: the master is transparent while $C = 0$, the slave while $C = 1$, so the value passes through only at the $0 \to 1$ transition.

Latch vs flip-flop

A latch is *level*-sensitive (transparent for the whole time the clock is high). A flip-flop is *edge*-triggered. Using latches in a loop with combinational logic can cause a value to race around the loop several times in one cycle; flip-flops prevent this.

Timing constraints. The input must be stable for a *setup time* before the edge and a *hold time* after it; the output changes a *clock-to-Q* delay after the edge. Therefore

$$T_{clock} \geq t_{clk \to Q} + t_{\text{critical path}} + t_{setup}.$$

4.2.1 Other flip-flops

- **JK**: like SR, but $J = K = 1$ toggles.
- **T**: toggles when $T = 1$, holds when $T = 0$. Useful for counters.

4.3 Registers and Register Files

An n-bit **register** is n D flip-flops sharing a clock. Adding a **write enable**: a mux in front of each flip-flop feeds back Q when the enable is 0, so the register keeps its value.

Definition 4.2. The MIPS **register file** holds 32 registers of 32 bits. It has

- two read ports: 5-bit read register numbers select outputs via two 32-to-1 muxes (of 32-bit buses);
- one write port: a 5-bit write register number goes through a 5-to-32 decoder, ANDed with `RegWrite`, to enable exactly one register; the write data is presented to all registers.

Reads are combinational; the write happens at the clock edge.

Other register types: **shift registers** (each flip-flop's input is its neighbour's output) and **counters** (register + incrementer).

4.4 Finite State Machines

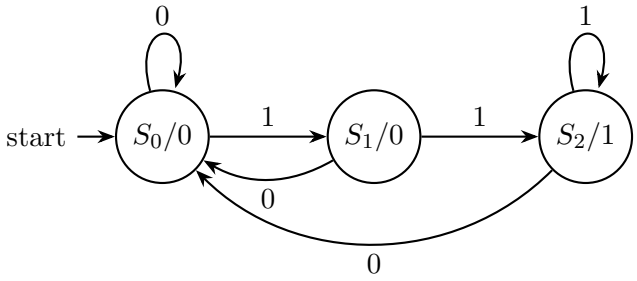
Definition 4.3. A **finite state machine (FSM)** consists of a state register, next-state logic (combinational; inputs: current state and inputs) and output logic.

- **Moore** machine: outputs depend only on the current state.
- **Mealy** machine: outputs depend on state and inputs.

Design procedure:

1. Draw the state diagram.
2. Choose a state encoding (binary, or one-hot with one flip-flop per state).
3. Write the state transition table and the output table.
4. Derive next-state and output equations (K-maps).
5. Build with flip-flops and gates.

Example 4.4 (Sequence detector)**.** Output 1 when the last two input bits were both 1 (Moore machine). States: S_0 (last bit 0 / start), S_1 (seen one 1), S_2 (seen $\geq$ two 1s, output 1).



With encoding $S_0 = 00$, $S_1 = 01$, $S_2 = 10$ (state bits Q_1Q_0, input x):

Q_1	Q_0	x	D_1	D_0	z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	1
1	0	1	1	0	1

State 11 is unused (don't care). Using don't cares: $D_1 = x(Q_1 + Q_0)$, $D_0 = x\overline{Q_1}\,\overline{Q_0}$, $z = Q_1$.

FSMs reappear as the control unit of the multi-cycle processor.

4.5 Memory

- **SRAM**: each bit is a latch-like cell (6 transistors). Fast, no refresh, expensive: used for caches and register files.
- **DRAM**: each bit is a capacitor + 1 transistor. Dense and cheap but slower, and must be periodically refreshed because charge leaks: used for main memory.

A memory with 2^k words is addressed through a decoder (row select) and muxes (column select).

Summary

- Feedback gives memory: SR latch $\rightarrow$ D latch (level-sensitive) $\rightarrow$ D flip-flop (edge-triggered, master–slave).
- Clock period $\geq$ clk-to-Q + critical path + setup.
- Register = flip-flops + write enable; register file = 32 registers, 2 read ports (muxes), 1 write port (decoder).
- FSM = state register + next-state logic + output logic; Moore vs Mealy; design via state table and K-maps.
- SRAM fast/expensive (caches), DRAM dense/slow (main memory).

Chapter 5

Data Representation and Manipulation

Everything in a computer is a string of bits. What those bits *mean* (an unsigned number, a signed number, a character, an instruction, a floating-point value) depends entirely on how they are interpreted.

5.1 Number Bases

A string $b_{n-1} \dots b_1 b_0$ in base r has value $\sum_{i=0}^{n-1} b_i r^i$.

- **Binary → decimal**: sum the powers of two. $1011_2 = 8 + 2 + 1 = 11$.
- **Decimal → binary**: repeatedly divide by 2 and read the remainders bottom-up, or subtract the largest powers of 2.
- **Hexadecimal** (base 16, digits 0–9, A–F): each hex digit is exactly 4 bits, so conversion is grouping. $1101\,0110\,1111_2 = \texttt{0xD6F}$.
- Octal groups 3 bits.

Example 5.1. $157 = 128 + 16 + 8 + 4 + 1 = 1001\,1101_2 = \texttt{0x9D}$.

Useful powers: $2^{10} \approx 10^3$ (Ki), 2^{20} (Mi), 2^{30} (Gi), $2^{32} \approx 4.29 \times 10^9$.

5.2 Unsigned Integers

With n bits, unsigned integers range over 0 to $2^n - 1$. Addition is done column by column with carries. If there is a carry out of the most significant bit, the true result does not fit: **unsigned overflow** (the stored result is the true sum mod 2^n).

5.3 Signed Integers

5.3.1 Sign–magnitude and one's complement

Sign–magnitude: MSB is the sign, rest is magnitude. Has two zeros (+0, −0) and needs special adder logic. **One's complement**: negate by flipping all bits; still two zeros, end-around carry needed. Both are historical.

5.3.2 Two's complement

Definition 5.2. In n-bit **two's complement**, the MSB has weight -2^{n-1}:

$$\text{value}(b_{n-1} \dots b_0) = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

The range is -2^{n-1} to $2^{n-1} - 1$ (for 32 bits: about ± 2.1 billion).

- **Negation**: invert all bits and add 1. (Shortcut: copy bits from the right up to and including the first 1, then invert the rest.)
- There is exactly one zero. The most negative number -2^{n-1} has no positive counterpart (negating it gives itself).
- Addition and subtraction use the *same* hardware as unsigned arithmetic; only the overflow rule differs.

Example 5.3. In 8 bits: $5 = 0000\,0101$, so $-5 = 1111\,1010 + 1 = 1111\,1011$. Check: $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$.

5.3.3 Sign extension

To widen an n-bit two's complement number to $m > n$ bits, copy the sign bit into the new high bits. 1011 (-5 in 4 bits) becomes 1111 1011 (-5 in 8 bits). MIPS sign-extends the 16-bit immediate of `addi`, `lw`, `sw`, `beq`. Zero extension (fill with 0s) is used for unsigned values and for logical immediates (`andi`, `ori`).

5.3.4 Signed overflow

Overflow rules

For two's complement addition $A + B$:

- overflow is impossible if A and B have different signs;
- overflow occurred iff A and B have the same sign and the result has the opposite sign;
- equivalently, carry into MSB $\neq$ carry out of MSB.

For $A - B$, overflow iff A and B have different signs and the result's sign differs from A's.

Example 5.4. 4-bit: $0111 + 0001 = 1000$, i.e. $7 + 1 = -8$: overflow (two positives gave a negative). $1111 + 0001 = 0000$ ($-1 + 1 = 0$): carry out of MSB but *no* signed overflow.

In MIPS, `add`, `addi`, `sub` trap (raise an exception) on signed overflow; `addu`, `addiu`, `subu` ignore it. C compilers use the unsigned forms.

5.4 Multiplication and Division

Multiplication. Shift-and-add, exactly like long multiplication: for each 1 bit of the multiplier, add the shifted multiplicand. Product of two n-bit numbers needs $2n$ bits. MIPS `mult $s, $t` puts the 64-bit product into the special registers `HI` (upper) and `LO` (lower), read with `mfhi`/`mflo`. Multiplying by 2^k is a left shift by k.

Division. Shift-and-subtract (restoring division). `div $s, $t` puts the quotient in `LO` and remainder in `HI`. Signed division: divide magnitudes, then the quotient is negative if the signs differ, and the remainder has the sign of the dividend.

5.5 Shifts and Logical Operations

- `sll` (logical left): fill with 0s; multiplies by 2^k.
- `srl` (logical right): fill with 0s; unsigned divide by 2^k.
- `sra` (arithmetic right): fill with copies of the sign bit; signed divide by 2^k (rounding toward $-\infty$).
- Bitwise `and` with a mask extracts bits; `or` sets bits; `xor` toggles bits; `nor $d, $s, $0` gives NOT.

5.6 Floating Point: IEEE 754

Real numbers are stored in binary scientific notation $(-1)^s \times 1.f \times 2^e$. The leading 1 is not stored (*hidden bit*) since normalized numbers always have it.

Format	sign	exponent	fraction	bias
Single (32-bit)	1	8	23	127
Double (64-bit)	1	11	52	1023

$$\text{value} = (-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-\text{Bias}}.$$

The exponent is stored **biased** (as unsigned $E = e + 127$) so that floats can be compared like unsigned integers.

5.6.1 Special values (single precision)

Exponent	Fraction	Meaning
0	0	± 0
0	$\neq 0$	denormalized: $(-1)^S \times 0.f \times 2^{-126}$
1–254	any	normalized number
255	0	$\pm\infty$
255	$\neq 0$	NaN (not a number)

Range of normalized singles: about $\pm 1.2 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$, with about 7 decimal digits of precision (double: about 16 digits).

Example 5.5 (Decimal to IEEE single)**.** Encode -0.75. $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$. So $S = 1$, $E = -1 + 127 = 126 = 0111\,1110$, fraction $= 100\ldots0$.

$$1\ 01111110\ 10000000000000000000000 = \texttt{0xBF400000}.$$

Example 5.6 (IEEE single to decimal)**.** `0x41480000` $= 0\ 10000010\ 10010000\ldots$. $S = 0$, $E = 130$ so $e = 3$, significand $= 1.1001_2 = 1.5625$. Value $= 1.5625 \times 8 = 12.5$.

5.6.2 Floating-point arithmetic

Addition: (1) align binary points by shifting the smaller number's significand right until exponents match; (2) add significands; (3) normalize (shift and adjust exponent, check over/underflow); (4) round (IEEE uses guard, round and sticky bits; default round-to-nearest-even). **Multiplication**: add exponents (subtracting one bias), multiply significands, normalize, round, sign = XOR of signs.

Pitfalls

Floating-point addition is *not associative*: $(-10^{38} + 10^{38}) + 1 = 1$ but $-10^{38} + (10^{38} + 1) = 0$. Many decimal fractions (e.g. 0.1) have no exact binary representation. Never compare floats for exact equality.

5.7 Characters

- **ASCII**: 7 bits (stored in a byte). `'0'` = 48 (0x30), `'A'` = 65 (0x41), `'a'` = 97 (0x61). Upper and lower case differ by 32 (bit 5).
- **Unicode**: up to 21-bit code points; UTF-8 encodes them in 1–4 bytes and is backward compatible with ASCII; UTF-16 uses 2 or 4.
- Strings in C are null-terminated byte arrays. MIPS has `lb`, `lbu`, `sb` for bytes (`lb` sign-extends, `lbu` zero-extends).

5.8 Endianness

A word occupies 4 bytes at addresses $a, a+1, a+2, a+3$. **Big-endian** puts the most significant byte at the lowest address (MIPS, network order); **little-endian** puts the least significant byte first (x86, ARM usually). Storing `0x12345678` at address 0: big-endian memory reads `12 34 56 78`, little-endian `78 56 34 12`.

Summary

- Hex = groups of 4 bits. Unsigned range $[0, 2^n - 1]$; two's complement range $[-2^{n-1}, 2^{n-1} - 1]$.
- Negate: invert and add 1. Sign-extend by copying the MSB.
- Signed overflow: same-sign operands, opposite-sign result ($C_{in,MSB} \neq C_{out,MSB}$). Unsigned overflow: carry out.
- IEEE 754 single: 1/8/23 bits, bias 127, hidden 1, special values for 0, denormals, ∞, NaN. FP addition is not associative.
- ASCII characters in bytes; endianness decides byte order in a word.

Chapter 6

The MIPS Instruction Set in Depth

With the five-instruction subset understood, we now look at the full MIPS ISA: register conventions, the three instruction formats, addressing modes, control flow and procedure calls.

6.1 Design Principles

Patterson and Hennessy summarize MIPS with four principles:

1. **Simplicity favours regularity**: all instructions are 32 bits, few formats, fields in the same place.
2. **Smaller is faster**: only 32 registers.
3. **Make the common case fast**: immediates for small constants, PC-relative branches.
4. **Good design demands good compromises**: three formats instead of one, to fit large constants and addresses.

6.2 Registers and Conventions

Number	Name	Use
0	`$zero`	constant 0
1	`$at`	reserved for the assembler
2–3	`$v0`–`$v1`	return values
4–7	`$a0`–`$a3`	arguments
8–15, 24–25	`$t0`–`$t9`	temporaries (caller-saved)
16–23	`$s0`–`$s7`	saved (callee-saved)
26–27	`$k0`–`$k1`	reserved for the OS kernel
28	`$gp`	global pointer
29	`$sp`	stack pointer
30	`$fp`	frame pointer
31	`$ra`	return address

6.3 Instruction Formats

	31–26	25–21	20–16	15–11	10–6	5–0
R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	immediate (16)		
J	op	address (26)				

- **R-format**: op = 0; the ALU operation is chosen by `funct`. `rs`, `rt` are sources, `rd` destination, `shamt` the shift amount for `sll/srl/sra`.
- **I-format**: arithmetic with immediates, loads/stores, branches. `rt` is the destination for `addi` and `lw`, and a source for `sw` and `beq`.
- **J-format**: `j` and `jal`.

6.3.1 Common instructions

Category	Instructions	Notes
Arithmetic	`add, addu, sub, subu, addi, addiu`	`u`: no overflow trap
Multiply/divide	`mult, multu, div, divu, mfhi, mflo`	HI/LO registers
Logical	`and, or, xor, nor, andi, ori, xori`	immediates zero-extended
Shifts	`sll, srl, sra, sllv, srlv, srav`	
Constants	`lui`	load upper 16 bits
Compare	`slt, sltu, slti, sltiu`	set to 1 if less than
Memory	`lw, sw, lb, lbu, sb, lh, lhu, sh`	
Branch	`beq, bne`	PC-relative
Jump	`j, jal, jr, jalr`	

6.3.2 Large constants

A 16-bit immediate cannot hold a 32-bit constant. Use `lui` to load the upper half and `ori` to fill in the lower half:

```
lui $t0, 0x1234        # $t0 = 0x12340000
ori $t0, $t0, 0x5678   # $t0 = 0x12345678
```

The pseudo-instruction `li $t0, 0x12345678` expands to exactly this (using `$at` where necessary). Other pseudo-instructions: `move`, `blt`, `bgt`, `la`.

6.4 Addressing Modes

1. **Register**: operand in a register (`add`).
2. **Immediate**: operand in the instruction (`addi`).
3. **Base/displacement**: memory at register + offset (`lw`).
4. **PC-relative**: branch target $= PC + 4 + (\text{sign-ext}(imm) \ll 2)$. Reach: $\pm 2^{15}$ instructions.
5. **Pseudo-direct**: jump target $= (PC + 4)[31{:}28] \,\|\, \text{address} \,\|\, 00$. The 26-bit field plus two implied zero bits covers 256 MB within the current region.

Example 6.1 (Encoding a branch)**.** At address 0x00400010: `beq $t0, $t1, L` where `L` is at 0x00400024. Offset $= (0x24 - 0x14)/4 = 4$. Encoding: op 4, rs 8, rt 9, imm 4: `000100 01000 01001 0000000000000100` = `0x11090004`.

Example 6.2 (Encoding a jump)**.** `j 0x00400040`: address field = 0x00400040 $\gg 2$ = 0x0100010. Encoding: `000010` followed by 26 bits of `0x0100010`, giving `0x08100010`.

6.5 Control Flow

Example 6.3 (If-then-else)**.** `if (i == j) f = g + h; else f = g - h;` with $f..j$ in `$s0..$s4`:

```
      bne  $s3, $s4, Else    # branch on the opposite condition
      add  $s0, $s1, $s2
      j    Exit
Else: sub  $s0, $s1, $s2
Exit:
```

Example 6.4 (Less-than branch)**.** `if (a < b) goto L;` with `a, b` in `$s0, $s1`:

```
slt $t0, $s0, $s1    # $t0 = (a < b)
bne $t0, $zero, L
```

MIPS has no `blt` in hardware: comparison with `slt` plus `beq`/`bne` against zero keeps the branch hardware simple.

Example 6.5 (While loop over an array)**.** `while (save[i] == k) i += 1;` with `i` in `$s3`, `k` in `$s5`, base of `save` in `$s6`:

```
Loop: sll  $t1, $s3, 2       # t1 = 4*i
      add  $t1, $t1, $s6     # t1 = &save[i]
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit:
```

6.6 Procedures

Steps of a procedure call:

1. Caller puts arguments in `$a0`–`$a3` (extra on the stack).
2. `jal Proc`: saves $PC + 4$ in `$ra` and jumps.
3. Callee allocates a stack frame, saves registers it must preserve, does its work, puts results in `$v0`/`$v1`.
4. Callee restores registers, frees the frame, `jr $ra`.

6.6.1 The stack

The stack lives at the top of memory and grows *downward*. `$sp` points to the top (last used word). Push: decrement `$sp`, then store. Pop: load, then increment.

Calling convention

- **Callee-saved**: `$s0`–`$s7`, `$sp`, `$fp`, `$ra` (the callee must restore them if it changes them; a non-leaf procedure must save `$ra` since its own `jal` overwrites it).
- **Caller-saved**: `$t0`–`$t9`, `$a0`–`$a3`, `$v0`–`$v1` (the callee may destroy them; the caller saves any it needs after the call).

Example 6.6 (Recursive factorial).

```
fact: addi $sp, $sp, -8       # make room for 2 words
      sw   $ra, 4($sp)        # save return address
      sw   $a0, 0($sp)        # save n
      slti $t0, $a0, 1        # n < 1 ?
      beq  $t0, $zero, L1
      addi $v0, $zero, 1      # base case: return 1
      addi $sp, $sp, 8        # pop (nothing changed)
      jr   $ra
L1:   addi $a0, $a0, -1
      jal  fact               # v0 = fact(n-1)
      lw   $a0, 0($sp)        # restore n
      lw   $ra, 4($sp)        # restore return address
      addi $sp, $sp, 8
      mul  $v0, $a0, $v0      # n * fact(n-1)
      jr   $ra
```

(`mul` is a pseudo-instruction for `mult` + `mflo`.)

6.6.2 Memory layout

From low to high addresses: reserved, **text** (code, starting at 0x00400000), **static data** (globals, accessed via `$gp`), **heap** (dynamic, grows up), and the **stack** (grows down from 0x7FFFFFFC). Heap and stack grow toward each other.

6.7 From Source to Execution

Compiler (C → assembly) → assembler (assembly → object file; expands pseudo-instructions, builds symbol and relocation tables) → linker (combines object files, resolves labels, fixes absolute addresses) → loader (copies the executable into memory, sets up stack and registers, jumps to the start).

6.8 RISC vs CISC

MIPS is RISC: fixed-length instructions, load/store, many registers, simple addressing. CISC ISAs (x86) have variable-length instructions, memory operands in arithmetic instructions and complex addressing. RISC makes the hardware simpler and pipelining easier; modern x86 chips internally translate instructions into RISC-like micro-operations.

Summary

- 32 registers with naming/usage conventions; `$zero` is hardwired.
- R/I/J formats; opcode in bits 31–26; R-type uses `funct`.
- Five addressing modes; branches are PC-relative (word offset from $PC+4$); jumps are pseudo-direct.
- `lui`/`ori` build 32-bit constants; `slt` + branch builds comparisons.
- Procedures: `jal`/`jr $ra`, arguments in `$a`, results in `$v`, stack grows down, callee saves `$s` and `$ra`.

Chapter 7

The Single-Cycle Datapath and Control

We now build a processor that executes `add`, `sub`, `and`, `or`, `slt`, `lw`, `sw`, `beq` (and later `j`) with each instruction completing in exactly one (long) clock cycle.

7.1 Performance

Definition 7.1. CPU time = Instruction count × CPI × Clock cycle time = $\dfrac{\text{IC} \times \text{CPI}}{\text{Clock rate}}$, where CPI is the average number of clock cycles per instruction.

The "iron law" shows the trade-offs: the ISA and compiler affect IC, the implementation affects CPI and cycle time. A single-cycle design has CPI = 1 but a long cycle time.

7.2 Datapath Elements

- **PC** register and an adder computing $PC + 4$.
- **Instruction memory**: address in, instruction out (read-only, combinational).
- **Register file**: two read ports, one write port (written at the clock edge when `RegWrite` = 1).
- **ALU** with `Zero` output.
- **Data memory**: address, write data, read data; `MemRead`/`MemWrite` controls.
- **Sign extender** (16 → 32 bits) and a **shift-left-2** unit for branch offsets, plus a second adder for the branch target.

Separate instruction and data memories are needed because a single-cycle design cannot use one memory twice in the same cycle (a structural hazard).

7.3 Building the Datapath

Fetch. Instruction ← IMem[PC]; PC ← PC + 4 (unless a branch is taken).

R-type. Read `rs`, `rt`; ALU computes; write result to `rd`.

Load/store. Read `rs` (base) and, for `sw`, `rt`; ALU adds base + sign-extended offset; `lw` reads data memory and writes to `rt`; `sw` writes `rt`'s value to memory.

Branch. ALU subtracts `rs` − `rt`; if `Zero`, the next PC is $PC + 4 + (\text{signext}(imm) \ll 2)$.

Merging these requires multiplexers wherever a unit has more than one possible source:

- `RegDst` mux: write register is `rt` (I-type) or `rd` (R-type).
- `ALUSrc` mux: second ALU operand is register `rt` or the sign-extended immediate.
- `MemtoReg` mux: write-back value is ALU result or memory data.
- `PCSrc` mux: next PC is $PC + 4$ or the branch target; `PCSrc` = `Branch` · `Zero`.

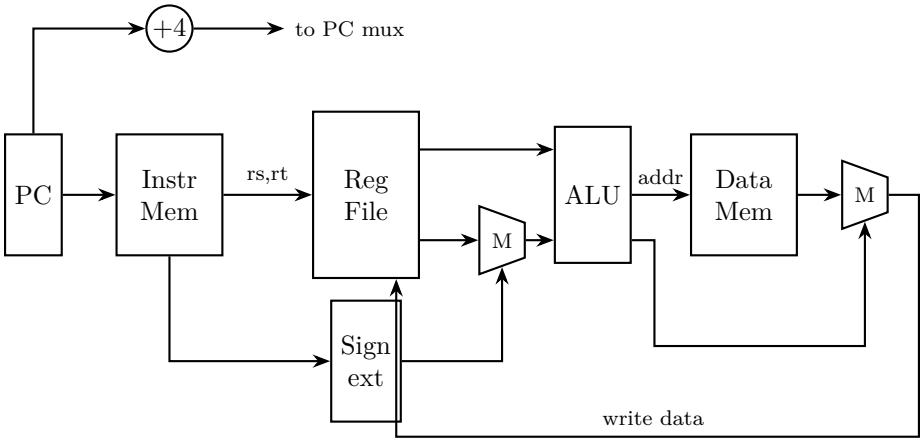


Figure 7.1: Simplified single-cycle datapath (branch logic and RegDst mux omitted for clarity).

7.4 Control

The **main control unit** decodes the 6-bit opcode into the control signals. A separate small **ALU control** unit combines a 2-bit `ALUOp` from main control with the `funct` field.

7.4.1 Main control table

Instr	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	1	0	0	1	0	0	0	10
`lw`	0	1	1	1	1	0	0	00
`sw`	X	1	X	0	0	1	0	00
`beq`	X	0	X	0	0	0	1	01

X means don't care: `sw` and `beq` do not write a register, so it does not matter which register number or write value is selected. (But `RegWrite` and `MemWrite` must never be X!)

7.4.2 ALU control

ALUOp	funct	Operation	ALU control
00 (lw/sw)	XXXXXX	add	0010
01 (beq)	XXXXXX	subtract	0110
10 (R)	100000	add	0010
10 (R)	100010	subtract	0110
10 (R)	100100	AND	0000
10 (R)	100101	OR	0001
10 (R)	101010	slt	0111

This two-level decoding (main control + ALU control) keeps the main control small and fast.

7.4.3 Adding jump

For `j`: the target is $\{(PC+4)[31{:}28], \text{addr}[25{:}0], 00\}$. Add one more mux after the `PCSrc` mux, controlled by a new `Jump` signal from main control.

Example 7.2 (Tracing `lw $t1, 8($t0)`).

1. Fetch from IMem[PC]; adder computes PC+4.
2. Register file reads `$t0` (rs); `rt` is `$t1`.
3. Sign-extend 8; `ALUSrc`=1 selects it; ALU control = add; ALU outputs `$t0`+8.
4. Data memory reads that address (`MemRead`=1).
5. `MemtoReg`=1 selects memory data; `RegDst`=0 selects `rt`; `RegWrite`=1 writes `$t1` at the clock edge.
6. `Branch`=0, so PC ← PC+4.

Exam-style question: control faults

A common question: "suppose `RegDst` is stuck at 0; which instructions break?" Answer: R-type instructions (they would write `rt` instead of `rd`); `lw` still works; `sw`, `beq` don't care. Always reason per instruction using the control table.

7.5 Timing of the Single-Cycle Design

The clock period must accommodate the slowest instruction, which is `lw`:

$$T = t_{IMem} + t_{RegRead} + t_{ALU} + t_{DMem} + t_{RegWrite\,setup}.$$

Example 7.3. Let IMem = 200 ps, register read = 100 ps, ALU = 200 ps, DMem = 200 ps, register write = 100 ps (ignore muxes). Instruction times: R-type 600, `lw` 800, `sw` 700, `beq` 500 ps. The single-cycle clock must be 800 ps even though most instructions need less. With a mix of 25% `lw`, 10% `sw`, 45% R-type, 20% `beq`, a hypothetical variable-length clock would average $0.25(800) + 0.10(700) + 0.45(600) + 0.20(500) = 640$ ps, so single-cycle wastes $800/640 = 1.25\times$.

Problems with single-cycle

- Cycle time set by the slowest instruction; common, fast instructions are penalized (violates "make the common case fast").
- Each functional unit used at most once per instruction, so units must be duplicated (two memories, three adders).
- Complex instructions (floating point) would make the cycle huge.

Solutions: multi-cycle (next chapter) and pipelining.

Summary

- CPU time = IC × CPI × cycle time.
- Datapath: PC, IMem, register file, ALU, DMem, sign extend, adders; muxes RegDst, ALUSrc, MemtoReg, PCSrc (and Jump).
- Control: main control from opcode (table above) + ALU control from ALUOp and funct.
- Single-cycle: CPI = 1, cycle time = longest instruction (`lw`).

Chapter 8

The Multi-Cycle Datapath

The single-cycle processor wastes time because every instruction takes as long as the slowest. The multi-cycle approach breaks each instruction into steps, each taking one *short* clock cycle; different instructions use different numbers of cycles.

8.1 Key Ideas

- Choose the clock period to fit the slowest *step* (roughly one major unit: a memory access, a register file access, or an ALU operation), not the slowest instruction.
- A functional unit can be reused in different cycles of the same instruction. So we need only **one memory** (for both instructions and data) and **one ALU** (it also computes $PC + 4$ and the branch target).
- Values produced in one cycle and needed in a later one must be saved in new **internal registers** at the end of the cycle:
  - **IR** (instruction register) and **MDR** (memory data register),
  - **A** and **B** (register file outputs),
  - **ALUOut** (ALU result).
- Extra muxes: the memory address comes from PC or ALUOut (`IorD`); ALU input A is PC or register A (`ALUSrcA`); ALU input B is register B, 4, the sign-extended immediate, or the shifted immediate (`ALUSrcB`, 2 bits); the new PC comes from ALU, ALUOut or the jump address (`PCSource`).
- Control becomes a **finite state machine**.

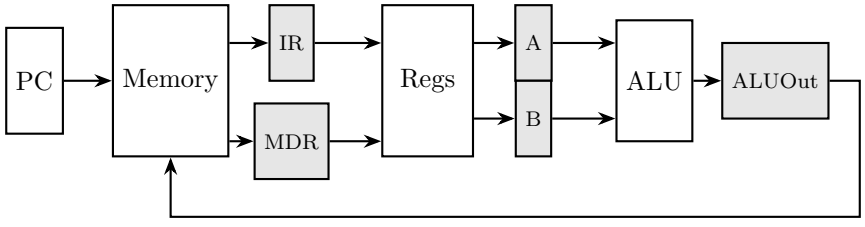


Figure 8.1: Sketch of the multi-cycle datapath: one memory, one ALU, and internal registers (shaded) between the stages. Muxes are omitted.

8.2 The Five Steps

Step	R-type	`lw`/`sw`	`beq`	`j`
1. Fetch	IR ← Mem[PC]; PC ← PC + 4			
2. Decode	A ← Reg[rs]; B ← Reg[rt]; ALUOut ← PC + (signext(imm) ≪ 2)			
3. Execute	ALUOut ← A op B	ALUOut ← A + signext(imm)	if (A == B) PC ← ALUOut	PC ← {PC[31:28], addr, 00}
4. Memory / R-completion	Reg[rd] ← ALUOut	`lw`: MDR ← Mem[ALUOut]; `sw`: Mem[ALUOut] ← B		
5. Write-back		`lw`: Reg[rt] ← MDR		

Cycle counts: `lw` 5, `sw` 4, R-type 4, `beq` 3, `j` 3.

Remark. In the decode step we compute the branch target *speculatively*: the ALU is idle anyway, and we do not yet know whether the instruction is a branch. If it is not, the result is simply ignored. This is an early example of doing work optimistically.

8.3 Control as an FSM

The control unit is a Moore FSM whose states correspond to the steps above. States 0 (fetch) and 1 (decode) are shared by all instructions; from state 1 the next state depends on the opcode:

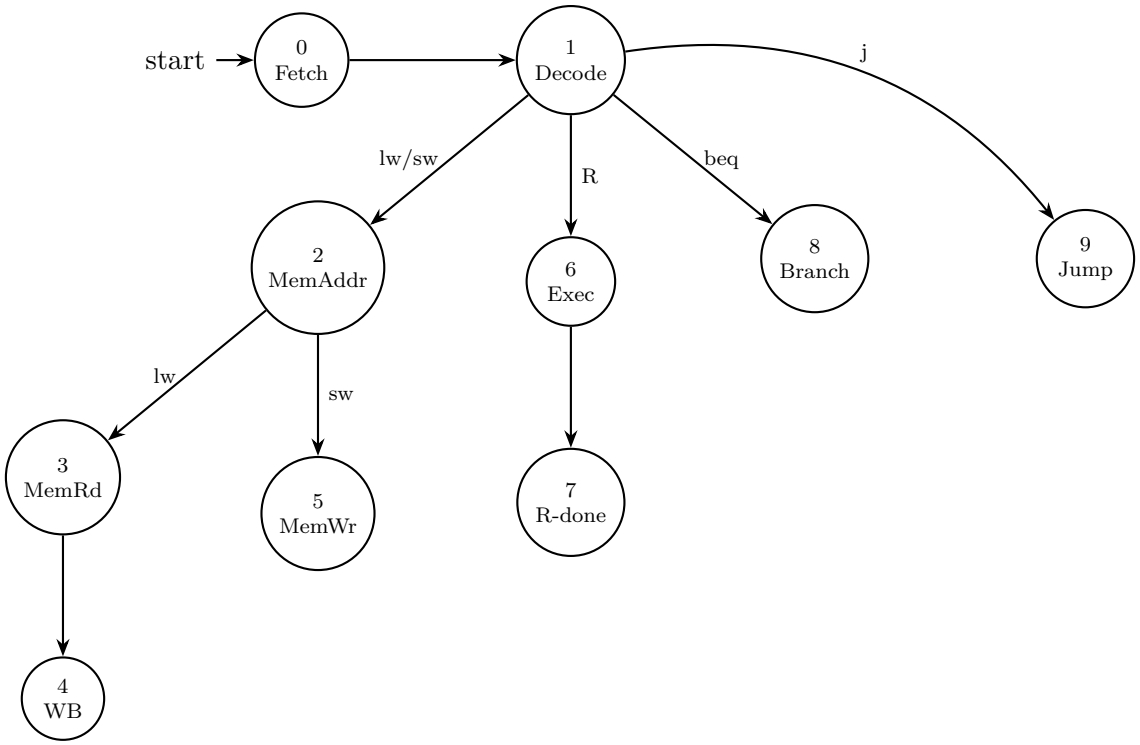


Figure 8.2: Multi-cycle control FSM. States 4, 5, 7, 8 and 9 all return to state 0 (arrows omitted).

State	Asserted signals (main ones)
0	MemRead, IRWrite, IorD=0, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCWrite, PCSource=00
1	ALUSrcA=0, ALUSrcB=11, ALUOp=00
2	ALUSrcA=1, ALUSrcB=10, ALUOp=00
3	MemRead, IorD=1
4	RegWrite, MemtoReg=1, RegDst=0
5	MemWrite, IorD=1
6	ALUSrcA=1, ALUSrcB=00, ALUOp=10
7	RegWrite, MemtoReg=0, RegDst=1
8	ALUSrcA=1, ALUSrcB=00, ALUOp=01, PCWriteCond, PCSource=01
9	PCWrite, PCSource=10

The PC is written when `PCWrite` + (`PCWriteCond` · `Zero`).

Implementing the FSM. Either as hardwired logic (a state register plus combinational next-state/output logic, e.g. a PLA or ROM), or by **microprogramming**: each state is a microinstruction stored in a control ROM, and a micro-PC sequences through them with dispatch tables for opcode-based branching. Microprogramming was popular for complex CISC instruction sets.

8.4 Performance Comparison

Example 8.1. Instruction mix: 25% `lw`, 10% `sw`, 45% R-type, 15% `beq`, 5% `j`. CPI = $0.25(5) + 0.10(4) + 0.45(4) + 0.15(3) + 0.05(3) = 1.25 + 0.4 + 1.8 + 0.45 + 0.15 = 4.05$.

If the multi-cycle clock is 200 ps (one memory access) and the single-cycle clock is 800 ps:

- single-cycle: $1 \times 800 = 800$ ps per instruction;
- multi-cycle: $4.05 \times 200 = 810$ ps per instruction.

Not a win here! The multi-cycle design pays overhead (register setup, uneven step lengths). Its real advantages are the hardware savings and the ability to have instructions of very different lengths (e.g. floating point divide) without slowing everything else.

Exam tip

To compare designs, always compute time per instruction (or total time) = CPI × cycle time, never CPI alone or clock rate alone.

8.5 Exceptions (brief)

An **exception** (internal, e.g. overflow or undefined opcode) or **interrupt** (external, e.g. I/O) transfers control to the OS. MIPS saves the faulting PC in the **EPC** register, records the cause in the **Cause** register and jumps to a fixed handler address (0x80000180). In the multi-cycle FSM this adds states that set EPC, Cause and the PC.

Summary

- Multi-cycle: short clock (one major unit per cycle), one memory, one ALU, internal registers IR, MDR, A, B, ALUOut.
- Steps: fetch, decode/register fetch, execute, memory, write-back; `lw` 5 cycles, R-type and `sw` 4, `beq` and `j` 3.
- Control is an FSM (hardwired or microprogrammed).
- Average CPI > 1; whether it beats single-cycle depends on the mix and the overhead. Pipelining is the real improvement.

Chapter 9

Pipelining and Hazards

Pipelining overlaps the execution of several instructions, like an assembly line or doing laundry: while one load is in the dryer, the next is in the washer. Each instruction still takes the same time (*latency*), but many more instructions complete per unit time (*throughput*).

9.1 The Five-Stage MIPS Pipeline

Stage	Name	Work
IF	Instruction fetch	read IMem[PC], PC ← PC+4
ID	Decode / register read	read registers, sign-extend, control
EX	Execute	ALU operation / address calculation
MEM	Memory	data memory read or write
WB	Write back	write result to register file

Pipeline registers **IF/ID**, **ID/EX**, **EX/MEM**, **MEM/WB** separate the stages and carry all data and control signals an instruction will need later (e.g. the destination register number must travel all the way to WB).

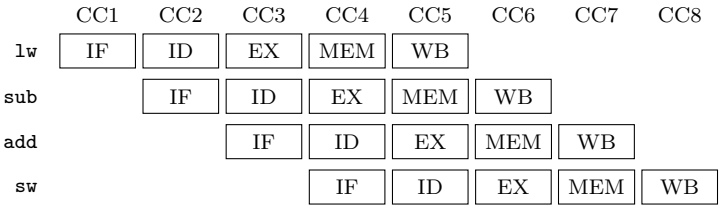


Figure 9.1: Pipeline diagram: four instructions overlap; one finishes every cycle once the pipeline is full.

9.1.1 Performance

- The clock period is set by the slowest *stage* (plus pipeline register overhead).
- Ideal CPI = 1 (one instruction completes per cycle in steady state).
- n instructions on a k-stage pipeline take $k + (n - 1)$ cycles.
- Ideal speedup over an unpipelined design ≈ number of stages, if stages are balanced.

Example 9.1. Stage times 200, 100, 200, 200, 100 ps. Single-cycle: 800 ps per instruction. Pipelined: clock 200 ps. For 1 000 000 instructions: single-cycle 8×10^8 ps; pipelined $(5 + 999\,999) \times 200 \approx 2 \times 10^8$ ps: speedup ≈ 4, not 5, because the stages are unbalanced.

MIPS is designed for pipelining: fixed-length instructions (easy fetch), few formats with source registers in fixed positions (decode in parallel with register read), memory operands only in loads/stores, aligned accesses.

9.2 Hazards

Definition 9.2. A **hazard** is a situation that prevents the next instruction from executing in its scheduled cycle.

- **Structural**: two instructions need the same hardware at once.
- **Data**: an instruction needs a value not yet produced.
- **Control**: the next instruction to fetch depends on a branch not yet resolved.

9.2.1 Structural hazards

With a single memory, IF of one instruction and MEM of another would conflict; MIPS avoids this with separate instruction and data memories (caches). The register file is written in the first half of a cycle and read in the second half, so WB and ID can share it.

9.2.2 Data hazards

```
sub $2, $1, $3     # writes $2 in WB (cycle 5)
and $12, $2, $5    # reads $2 in ID (cycle 3)  -- hazard
or  $13, $6, $2    # reads $2 in cycle 4       -- hazard
add $14, $2, $2    # reads in cycle 5: OK (write first half, read second half)
sw  $15, 100($2)   # OK
```

These are **read-after-write (RAW)** dependences. Without help we would need to **stall** (insert bubbles) for two cycles.

Forwarding (bypassing). The result of `sub` exists at the end of its EX stage, long before WB. A **forwarding unit** feeds it directly from the EX/MEM or MEM/WB pipeline register into the ALU inputs.

Forwarding conditions

EX hazard (forward from EX/MEM):

`EX/MEM.RegWrite` and `EX/MEM.Rd` $\neq 0$ and `EX/MEM.Rd` = `ID/EX.Rs` (or `ID/EX.Rt`).

MEM hazard (forward from MEM/WB): same with MEM/WB, *but only if* the EX hazard condition does not also hold for that operand (the more recent result wins).

Load-use hazard. A loaded value is only available at the end of MEM, so an instruction immediately using it cannot be helped by forwarding alone:

```
lw  $2, 20($1)
and $4, $2, $5     # needs $2 in EX, the same cycle lw is in MEM
```

The **hazard detection unit** in ID checks

$$\texttt{ID/EX.MemRead} \text{ and } (\texttt{ID/EX.Rt} = \texttt{IF/ID.Rs} \text{ or } \texttt{ID/EX.Rt} = \texttt{IF/ID.Rt})$$

and if true stalls one cycle: it prevents PC and IF/ID from updating and zeroes the control signals going into ID/EX (a bubble, i.e. a `nop`). After the one-cycle stall, forwarding from MEM/WB supplies the value.

Code scheduling. The compiler can reorder instructions to avoid load-use stalls.

Example 9.3. `a = b + e; c = b + f;`

```
# original: 2 stalls          # reordered: 0 stalls
lw  $t1, 0($t0)               # lw  $t1, 0($t0)
lw  $t2, 4($t0)               # lw  $t2, 4($t0)
add $t3, $t1, $t2  # stall    # lw  $t4, 8($t0)
sw  $t3, 12($t0)              # add $t3, $t1, $t2
lw  $t4, 8($t0)               # sw  $t3, 12($t0)
add $t5, $t1, $t4  # stall    # add $t5, $t1, $t4
sw  $t5, 16($t0)              # sw  $t5, 16($t0)
```

Other dependences. Write-after-read (WAR) and write-after-write (WAW) cannot cause hazards in the simple in-order 5-stage pipeline (all reads in ID, all writes in WB, in order) but matter in out-of-order processors.

9.2.3 Control hazards

A `beq`'s outcome is known only after comparing registers. If resolved in MEM (as in the basic design), three instructions have already been fetched.

Approaches:

1. **Stall** until the branch resolves (simple, slow).
2. **Resolve earlier**: move the comparison (an XOR-based equality tester) and target adder into ID, so only one instruction is fetched behind the branch. This may itself require forwarding into ID and stalls if the branch's operand is being computed just before it.
3. **Predict not taken**: keep fetching sequentially; if the branch is taken, **flush** the wrongly fetched instructions (turn them into nops).
4. **Delayed branch** (classic MIPS): the instruction right after the branch (the *delay slot*) always executes; the compiler fills it with useful work or a `nop`.
5. **Dynamic branch prediction**: hardware remembers past behaviour.

Dynamic prediction. A **branch history table** (branch prediction buffer) is indexed by low bits of the branch address.

- 1-bit predictor: remember the last outcome. A loop branch is mispredicted twice per execution of the loop (last iteration and first iteration next time).
- 2-bit saturating counter: change the prediction only after two consecutive mispredictions. A loop branch mispredicts once per loop.
- A **branch target buffer** caches target addresses so the target can be fetched in the next cycle.

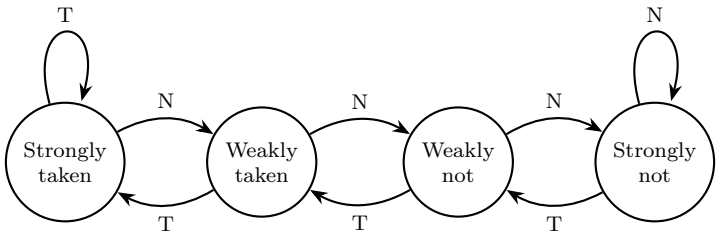


Figure 9.2: 2-bit saturating-counter branch predictor (T = taken, N = not taken).

Example 9.4 (CPI with hazards)**.** Base CPI 1. 20% of instructions are loads, half of which are followed by a dependent instruction (1-cycle stall). 15% are branches, predicted not-taken, 60% taken, penalty 1 cycle when mispredicted.

$$\text{CPI} = 1 + 0.20 \times 0.5 \times 1 + 0.15 \times 0.6 \times 1 = 1.19.$$

9.3 Beyond the Basic Pipeline

- **Deeper pipelines** (superpipelining): more, shorter stages, higher clock rate, but larger branch penalties.
- **Multiple issue**: issue several instructions per cycle. *Static* (VLIW: compiler bundles instructions) or *dynamic* (superscalar: hardware decides). Then CPI < 1; we talk about IPC.
- **Out-of-order execution** with register renaming removes WAR/WAW false dependences and hides latencies; results are committed in order via a reorder buffer.
- **Exceptions** in a pipeline: flush the faulting instruction and those after it, save its PC in EPC, fetch from the handler.

Summary

- Five stages IF, ID, EX, MEM, WB with pipeline registers between them; ideal CPI 1, clock set by slowest stage, speedup $\leq$ number of stages.
- Structural hazards: avoided by separate I/D memories and split-cycle register file.
- Data hazards: forwarding from EX/MEM and MEM/WB; load-use needs a 1-cycle stall (hazard detection unit); compiler scheduling helps.

- Control hazards: stall, early resolution in ID, predict not taken with flush, delay slots, dynamic prediction (2-bit counters, BTB).
- CPI = 1 + stall cycles per instruction.

Chapter 10

Memory Hierarchy and Caches

Processors are much faster than main memory (a DRAM access takes on the order of 100 processor cycles). We want memory that is as large as the cheapest technology and as fast as the fastest. The trick is a **memory hierarchy**: small fast memories close to the processor hold copies of the data most likely to be used next.

10.1 Locality

Definition 10.1. Temporal locality: an item referenced recently is likely to be referenced again soon (loop variables, instructions in a loop). **Spatial locality**: items near a recently referenced item are likely to be referenced soon (sequential instructions, array traversal).

Level	Technology	Typical access time	Typical size
Registers	flip-flops	< 1 ns	$\sim$100s of bytes
L1 cache	SRAM	$\sim$1 ns	32–64 KB
L2/L3 cache	SRAM	3–20 ns	MBs
Main memory	DRAM	50–100 ns	GBs
Disk / SSD	magnetic / flash	μs–ms	TBs

10.2 Terminology

- **Block** (line): unit of data copied between levels.
- **Hit**: the data is found in the upper level. **Hit rate** h = fraction of accesses that hit; **miss rate** $= 1 - h$.
- **Hit time**: time to access the upper level, including determining hit or miss.
- **Miss penalty**: time to fetch the block from the lower level and deliver it.

Average memory access time

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}.$$

For multiple levels: $\text{AMAT} = t_{L1} + m_{L1}(t_{L2} + m_{L2} \times t_{mem})$.

10.3 Direct-Mapped Caches

Each memory block maps to exactly one cache line: index = (block address) mod (number of lines). A byte address is split into fields:

Tag	Index	Block offset

With 2^n lines and 2^m-byte blocks on a 32-bit address: offset = m bits, index = n bits, tag = $32 - n - m$ bits. Each line stores a **valid bit**, the **tag**, and the data. A hit means the line at the index is valid and its tag matches the address's tag.

Example 10.2 (Field sizes)**.** A 16 KB direct-mapped cache with 16-byte blocks, 32-bit addresses. Lines = 16K/16 = 1024 = 2^{10}. Offset 4 bits, index 10 bits, tag $32 - 14 = 18$ bits. Total storage: $1024 \times (1 + 18 + 128) = 150\,528$ bits ≈ 18.4 KB, more than the 16 KB of data.

Example 10.3 (Tracing accesses)**.** 8-line direct-mapped cache, 1-word blocks, word addresses 22, 26, 22, 26, 16, 3, 16, 18. Index = address mod 8.

Addr	Binary	Index	Tag	Result
22	10110	110	10	miss (cold)
26	11010	010	11	miss (cold)
22	10110	110	10	hit
26	11010	010	11	hit
16	10000	000	10	miss (cold)
3	00011	011	00	miss (cold)
16	10000	000	10	hit
18	10010	010	10	miss (conflict: replaces 26)

Hit rate 3/8.

10.3.1 Block size

Larger blocks exploit spatial locality and lower the miss rate, up to a point: with a fixed cache size, too-large blocks mean too few lines (more conflicts), and the miss penalty grows (more data to transfer).

10.4 The Three Cs of Misses

- **Compulsory** (cold): first access to a block.
- **Capacity**: the working set does not fit in the cache.
- **Conflict**: too many blocks map to the same set, even though the cache has room elsewhere. Reduced by associativity.

10.5 Associative Caches

- **Fully associative**: a block can go in any line; compare all tags in parallel (expensive). No conflict misses.

- **k-way set associative**: lines grouped into sets of k; a block maps to one set (set = block address mod #sets) and can occupy any of the k lines there. Search k tags in parallel with k comparators and a mux.

Direct-mapped is 1-way; fully associative is N-way. For a fixed cache size, doubling associativity halves the number of sets: the index shrinks by one bit and the tag grows by one bit.

Replacement. In a set-associative cache, on a miss we must pick a victim: **LRU** (least recently used; easy for 2-way with one bit per set), random, or FIFO.

Example 10.4. Block addresses 0, 8, 0, 6, 8 on caches with four 1-block lines.

- Direct-mapped (index mod 4): 0 → 0 miss, 8 → 0 miss (evicts 0), 0 miss (evicts 8), 6 → 2 miss, 8 miss. 5 misses.
- 2-way (2 sets, LRU): 0 → set 0 miss, 8 → set 0 miss, 0 hit, 6 → set 0 miss (evicts 8, the LRU), 8 miss (evicts 0). 4 misses.
- Fully associative: 0, 8 miss, 0 hit, 6 miss, 8 hit. 3 misses.

10.6 Writes

- **Write-through**: every write updates both cache and memory. Simple, memory always consistent, but slow; use a **write buffer** so the processor need not wait.
- **Write-back**: write only the cache; mark the line **dirty**; write it to memory when it is evicted. Less memory traffic but more complex.

On a write miss: **write-allocate** (fetch the block into the cache, then write; usual with write-back) or **no-write-allocate** (write memory only; usual with write-through).

10.7 Cache Performance

$$\text{CPU time} = \text{IC} \times (\text{CPI}_{base} + \text{memory stall cycles per instruction}) \times \text{cycle time},$$

$$\text{Stall cycles per instr.} = \frac{\text{memory accesses}}{\text{instruction}} \times \text{miss rate} \times \text{miss penalty}.$$

Example 10.5. Base CPI 2, I-cache miss rate 2%, D-cache miss rate 4%, miss penalty 100 cycles, 36% of instructions are loads/stores. I-stalls $= 1 \times 0.02 \times 100 = 2$; D-stalls $= 0.36 \times 0.04 \times 100 = 1.44$. CPI $= 2 + 2 + 1.44 = 5.44$. A perfect cache would make the machine $5.44/2 = 2.72$ times faster.

Example 10.6 (AMAT)**.** Clock 1 ns, hit time 1 cycle, miss penalty 20 cycles, miss rate 5%. AMAT $= 1 + 0.05 \times 20 = 2$ cycles $= 2$ ns.

Multilevel caches. L1 is small and optimized for hit time; L2 (and L3) are larger and optimized for miss rate, reducing the effective miss penalty of L1. *Local* miss rate = misses / accesses to that level; *global* miss rate = misses / all CPU accesses.

Software. Access patterns matter: traversing a 2D array in the order it is stored (row-major in C) has good spatial locality; blocking (tiling) algorithms like matrix multiply keep the working set in cache.

Summary

- Locality (temporal, spatial) makes the hierarchy work.
- Address = tag | index | offset; valid bit + tag comparison decides a hit.
- Direct-mapped (1 place), k-way set associative, fully associative; LRU replacement.
- Misses: compulsory, capacity, conflict.
- Write-through (+ write buffer) vs write-back (dirty bit); write-allocate vs no-write-allocate.
- AMAT = hit time + miss rate $\times$ miss penalty; stall cycles add to CPI.

Chapter 11

Virtual Memory

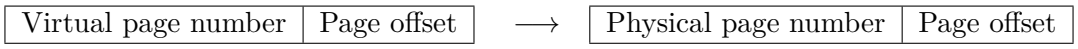
Virtual memory treats main memory as a "cache" for secondary storage (disk/SSD). Each program sees its own large, private **virtual address space**; hardware and the operating system translate virtual addresses to **physical addresses**.

11.1 Why Virtual Memory?

- **Protection and isolation**: a process cannot read or corrupt another process's memory.
- **Sharing**: several programs share physical memory safely; shared libraries can be mapped into many address spaces.
- **Illusion of large memory**: programs can be larger than physical memory; unused parts stay on disk.
- **Relocation**: a program can be loaded anywhere in physical memory without changing its addresses.

11.2 Pages and Address Translation

Memory is divided into fixed-size **pages** (typically 4 KB). The virtual address is split into a **virtual page number (VPN)** and a **page offset**; translation replaces the VPN by a **physical page number (PPN)** and keeps the offset.



Cache term	VM term
block	page
miss	page fault
index/tag lookup	page table lookup

11.3 Page Tables

Definition 11.1. A **page table** is an array in memory, indexed by VPN, whose entries (PTEs) contain a **valid** bit, the PPN (or the disk location if not resident), and status bits: **dirty**, **reference**

(used), and **protection** (read/write/execute, user/kernel). Each process has its own page table; the hardware **page table register** points to the current one.

Example 11.2 (Page table size)**.** 32-bit virtual addresses, 4 KB pages, 4-byte PTEs. Offset = 12 bits, VPN = 20 bits, so 2^{20} entries $\times$ 4 B = 4 MB per process. For 64-bit addresses this is far too big, so real systems use **multi-level (hierarchical) page tables**, where only the parts of the table actually in use are allocated, or inverted/hashed page tables.

11.4 Page Faults

If the PTE is invalid, a **page fault** exception occurs and the OS:

1. finds the page on disk;
2. chooses a physical page to replace (approximating LRU using reference bits, e.g. the clock algorithm); if the victim is dirty, writes it back to disk;
3. reads the needed page into memory and updates the page table;
4. restarts the faulting instruction.

Design consequences of huge miss penalties

A page fault costs millions of cycles, so:

- pages are large (amortize the disk access);
- placement is **fully associative** (any virtual page in any physical page) to minimize the miss rate;
- misses are handled in *software* (clever algorithms are worth it);
- always **write-back**, never write-through.

11.5 The TLB

Without help, every memory access would need an extra memory access to read the PTE, doubling the access time. The **translation lookaside buffer (TLB)** is a small cache of recent translations (typically 16–512 entries, often fully associative). Each entry holds a tag (VPN), PPN, and valid, dirty, reference and protection bits.

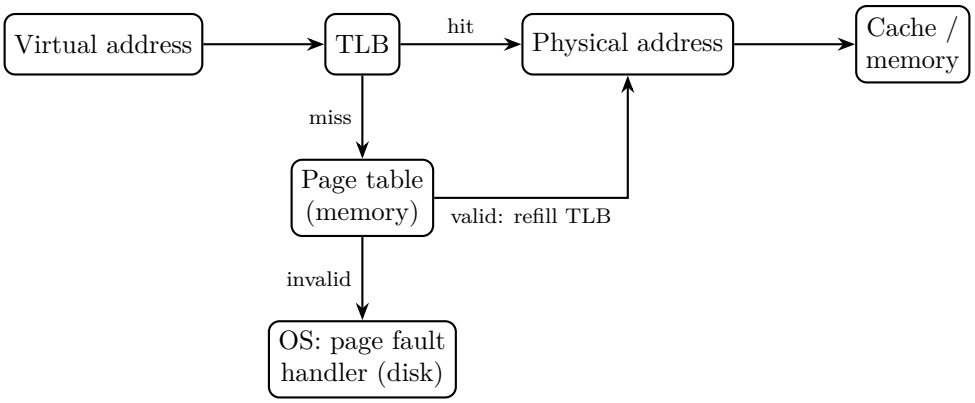


Figure 11.1: Translation flow.

Possible outcomes of an access:

TLB	Page table	Cache	Possible?
hit	hit	hit/miss	yes (best case: hit/hit/hit)
miss	hit	hit/miss	yes: refill TLB from page table
miss	miss	miss	yes: page fault
hit	miss	–	no: TLB only caches valid translations
miss	miss	hit	no: data not in memory cannot be in the cache

TLB misses can be handled in hardware (a page table walker, as in x86) or in software (an exception handler, as in MIPS).

11.6 Caches and Virtual Memory Together

- **Physically addressed cache**: translate first, then access the cache. Simple but the TLB is on the critical path.
- **Virtually indexed, physically tagged (VIPT)**: index the cache with page-offset bits (which do not change under translation) while the TLB translates in parallel; compare the physical tag afterwards. This works when index + offset bits fit in the page offset.
- Virtually addressed caches avoid translation but suffer from aliasing (two virtual addresses for the same physical data) and must be flushed or tagged with process IDs on context switch.

Example 11.3 (Address breakdown)**.** 32-bit virtual addresses, 30-bit physical addresses, 8 KB pages. Offset = 13 bits; VPN = 19 bits; PPN = 17 bits. A 4-entry fully associative TLB stores a 19-bit tag and a 17-bit PPN per entry (plus status bits).

11.7 Protection

The OS (running in kernel/supervisor mode) is the only one allowed to modify page tables and the page table register. User programs enter the kernel via **system calls** (`syscall` on MIPS), which switch modes. On a context switch the OS changes the page table register and invalidates (or tags with an address-space ID) the TLB.

Summary

- VM gives each process a private address space: protection, sharing, relocation, and the illusion of more memory.
- Virtual address = VPN | offset; page table maps VPN → PPN with valid, dirty, reference and protection bits.
- Page faults are very expensive: large pages, fully associative placement, LRU approximation, write-back, handled by the OS.
- The TLB caches translations; know which TLB/page table/cache combinations are possible.

Chapter 12

ARM: 32-bit and ARMv8

ARM (originally Acorn RISC Machine, later Advanced RISC Machines) is the most widely shipped ISA in the world: nearly every phone and tablet, and many embedded systems and laptops, use it. Like MIPS it is a RISC load/store architecture, so almost everything we learned carries over; the differences are instructive.

12.1 ARM 32-bit (ARMv7)

12.1.1 Registers

- 16 registers `r0`–`r15`, each 32 bits: `r13` = `sp` (stack pointer), `r14` = `lr` (link register, holds return address), `r15` = `pc` (the program counter is a visible general register!).
- No hardwired zero register.
- Calling convention: arguments/results in `r0`–`r3`, callee-saved `r4`–`r11`.
- A **current program status register (CPSR)** holds the condition flags **N** (negative), **Z** (zero), **C** (carry), **V** (overflow).

12.1.2 Condition codes and conditional execution

Instead of MIPS's compare-and-branch (`beq`, `slt`), ARM sets condition flags with `CMP` (or any instruction with the `S` suffix, e.g. `ADDS`) and then branches on them. Moreover, almost *every* 32-bit ARM instruction has a 4-bit condition field (bits 31–28): it executes only if the condition holds.

Suffix	Meaning	Flags
EQ / NE	equal / not equal	Z=1 / Z=0
LT / GE	signed less / greater-or-equal	N≠V / N=V
GT / LE	signed greater / less-or-equal	Z=0 and N=V / Z=1 or N≠V
LO / HS	unsigned lower / higher-or-same	C=0 / C=1
AL	always	–

Example 12.1 (Conditional execution avoids branches)**.** `if (a == b) c = a + b; else c = a - b;` with `a, b, c` in `r0, r1, r2`:

```
CMP   r0, r1
ADDEQ r2, r0, r1      ; executes only if Z = 1
SUBNE r2, r0, r1      ; executes only if Z = 0
```

No branches means no control hazards for short if/else bodies. MIPS needs a branch and a jump for the same code.

12.1.3 Flexible second operand

The second source operand of a data-processing instruction can be a register shifted or rotated by a constant or by another register, for free:

```
ADD r0, r1, r2, LSL #2   ; r0 = r1 + (r2 << 2)
```

Immediates are 12 bits: an 8-bit value rotated right by an even amount (so a constant such as `0xFF000000` fits, but not every 16-bit value).

12.1.4 Addressing modes

ARM has richer memory addressing than MIPS:

Mode	Example / meaning
Immediate offset	`LDR r0, [r1, #8]` : r0 = M[r1+8]
Register offset	`LDR r0, [r1, r2]` : r0 = M[r1+r2]
Scaled register	`LDR r0, [r1, r2, LSL #2]` : r0 = M[r1+4r2]
Pre-indexed (write-back)	`LDR r0, [r1, #4]!` : r1 += 4; r0 = M[r1]
Post-indexed	`LDR r0, [r1], #4` : r0 = M[r1]; r1 += 4
PC-relative	`LDR r0, label`

Auto-increment addressing makes array loops shorter. `LDM`/`STM` (load/store multiple) save or restore a list of registers in one instruction, convenient for procedure entry and exit:

```
PUSH {r4-r6, lr}      ; = STMDB sp!, {r4-r6, lr}
...
POP  {r4-r6, pc}      ; restore and return in one instruction
```

12.1.5 Procedures

`BL label` (branch and link) stores the return address in `lr`, like MIPS `jal`; return with `BX lr` or `MOV pc, lr`.

Example 12.2 (Array sum in ARM).

```
; r0 = address of array, r1 = n; result in r2
      MOV  r2, #0
loop: CMP  r1, #0
      BEQ  done
      LDR  r3, [r0], #4     ; r3 = *p; p += 4 (post-index)
      ADD  r2, r2, r3
      SUBS r1, r1, #1       ; also sets flags
      B    loop
done:
```

12.1.6 Thumb

Thumb is a 16-bit compressed encoding of common instructions for better code density (Thumb-2 mixes 16- and 32-bit instructions). MIPS has a similar extension (MIPS16).

12.2 Comparison: MIPS vs ARM 32-bit

	MIPS	ARMv7
Instruction size	32 bits	32 bits (16 in Thumb)
Registers	32, `$zero` hardwired	16, PC is `r15`
Conditional branches	compare registers (`beq`, `bne`, `slt`)	condition codes (NZCV) set by `CMP`
Conditional execution	no	nearly every instruction
Immediates	16 bits	12 bits (8-bit rotated)
Addressing modes	base + offset only	many, incl. auto-increment, scaled
Load/store multiple	no	yes
Shifts	separate instructions	folded into the second operand
Memory	aligned, big-endian (configurable)	usually little-endian

12.3 ARMv8 (AArch64)

ARMv8 (2011) introduced a 64-bit execution state, **AArch64**, with a new instruction set (A64) that is in many ways *closer to MIPS*:

- 31 general-purpose 64-bit registers `X0`–`X30` (`W0`–`W30` name the low 32 bits); register 31 acts as the zero register `XZR` or the stack pointer `SP` depending on context. `X30` is the link register.
- The PC is *no longer* a general register.
- Conditional execution of arbitrary instructions was *dropped* (it complicated out-of-order implementations); only a few conditional instructions remain (`CSEL`, conditional select).
- Compare-and-branch instructions `CBZ`/`CBNZ` (branch if register is zero/nonzero), similar to MIPS `beq $r, $zero`.
- Load/store multiple dropped, replaced by load/store pair (`LDP`/`STP`).
- Still fixed 32-bit instructions; still a load/store architecture.

Example 12.3 (ARMv8 procedure skeleton)**.**

```
func: STP  X29, X30, [SP, #-16]!   ; push frame pointer and link register
      MOV  X29, SP
      ADD  X0, X0, X1              ; compute result in X0
      LDP  X29, X30, [SP], #16     ; pop
      RET                          ; return via X30
```

Big lesson

ARMv8 shows the industry converging on MIPS-like RISC principles: lots of registers, a zero register, simple fixed-length encodings, compare-and-branch. Features that once saved instructions (conditional execution, PC as a general register, load-multiple) became obstacles to deep pipelining and out-of-order execution.

12.4 Fallacies and Pitfalls (course themes)

- *Fallacy*: more powerful instructions mean higher performance. Complex instructions are hard to pipeline.
- *Fallacy*: clock rate (or MIPS rating) alone measures performance. Use IC $\times$ CPI $\times$ cycle time.
- *Pitfall*: Amdahl's law: improving one part of a task by a factor s gives overall speedup $\frac{1}{(1-f)+f/s}$, where f is the fraction affected.

Summary

- ARMv7: 16 registers (PC visible), NZCV condition flags, conditional execution of almost every instruction, flexible shifted operands, rich addressing modes, load/store multiple, Thumb.
- MIPS: 32 registers, `$zero`, compare-in-branch, minimal addressing.
- ARMv8/AArch64: 31 64-bit registers + zero register, no general conditional execution, PC not a GPR, `CBZ`/`CBNZ`, `LDP`/`STP`: closer to MIPS.
- All are RISC load/store ISAs with fixed 32-bit instructions, so the datapath and pipelining ideas of this course apply directly.